SLA-2

PRN: 260840125033 260840125041

Q . In Big Data systems, unstructured data arrives continuously in huge volumes. A single thread cannot keep up with reading, parsing, and aggregating this data.
In this assignment, you will build a event tracker Engine that reads a batch of mock values (of counter), distributes the workload across a pool of managed workers

Understand :
1. Why is manually creating new Thread() inside a loop a terrible practice when dealing with millions of Big Data records? What resource exhaustion occurs?

Creating a separate Thread for every item in a very large data stream does not scale well. Each thread consumes memory for its stack and other runtime resources. A huge number of threads can exhaust native memory or operating-system thread limits and may lead to an OutOfMemoryError or failure to create a native thread. Large thread counts also increase scheduling and context-switching overhead. The CPU spends more time managing threads, and performance can fall instead of improving. For millions of records, tasks should normally be grouped into manageable jobs and executed by a controlled number of workers.

2. Research java.util.concurrent.ExecutorService and Executors.newFixedThreadPool(int). Explain how a thread pool optimizes CPU usage compared to creating raw threads.
Also explain what factors affect the selection of ThreadPool size

Executors.newFixedThreadPool(n) creates a pool that uses up to n worker threads. Submitted tasks wait in a queue when all workers are busy, and workers can process later tasks without creating a new thread for every task. This approach limits concurrent threads, reuses workers, reduces thread-creation overhead, and makes shutdown and waiting for completion easier to manage. A pool does not make work cost-free: an unbounded task queue can still consume memory if tasks arrive much faster than workers can complete them.

Factors that affect thread-pool size:

• CPU-bound work: for computation-heavy tasks, a starting point is around the number of available CPU cores; too many workers may increase context switching.
• I/O-bound work: tasks that spend time waiting for files, databases, or networks may benefit from more workers than CPU cores, within resource limits.
• Task duration and arrival rate: short tasks and high submission rates need careful queue and throughput planning.
• Memory: each active thread has stack and runtime overhead, and queued tasks also consume memory.
• External limits: database connection pools, API limits, file handles, and service capacity can constrain useful concurrency.
• Measurement: benchmark with realistic workloads and monitor CPU use, queue length, latency, and failures rather than relying on one universal number

Implement a basic computation job that increments a single shared counter, submit it to a thread pool, and safely shut the pool down.

1. A simple thread-safe counter representing total processed records
2. A Job that shows number of records processed
3. With Total_Jobs = 10 and records_per_job = 1000, Print the final pipeline result.

ProcessedCounter.java — thread-safe shared total

```
package sla2;
import java.util.concurrent.atomic.AtomicLong;
public class ProcessedCounter {
 private final AtomicLong total = new AtomicLong(0);
 public void addRecords(long amount) {
 total.addAndGet(amount);
 }
 public long getTotal() {
 return total.get();
 }
}
```

RecordJob.java — each job simulates processing its assigned records

```
package sla2;
public class RecordJob implements Runnable {
 private final int jobNumber;
 private final int recordCount;
 private final ProcessedCounter counter;
 public RecordJob(int jobNumber, int recordCount, ProcessedCounter counter) {
 this.jobNumber = jobNumber;
 this.recordCount = recordCount;
 this.counter = counter;
 }
 @Override
 public void run() {
 long processed = 0;
 // Simulate processing the records assigned to this job.
```

```
 for (int i = 0; i < recordCount; i++) {
 processed++;
 }
 counter.addRecords(processed);
 System.out.println("Job " + jobNumber + " processed " + processed + " records.");
 }
}
```

EventTrackerEngine.java — submits jobs and shuts down the pool

```
package sla2;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.TimeUnit;
public class EventTrackerEngine {
 private final int workerCount;
 public EventTrackerEngine(int workerCount) {
 this.workerCount = workerCount;
 }
 public void execute(int jobCount, int recordsPerJob) throws InterruptedException {
 ProcessedCounter counter = new ProcessedCounter();
 ExecutorService pool = Executors.newFixedThreadPool(workerCount);
 System.out.println("Starting event tracker with " + workerCount + " workers.");
 try {
 for (int job = 1; job <= jobCount; job++) {
 pool.submit(new RecordJob(job, recordsPerJob, counter));
 }
 } finally {
 // Stop accepting new tasks; submitted tasks can finish.
 pool.shutdown();
 }
 if (!pool.awaitTermination(30, TimeUnit.SECONDS)) {
 System.err.println("Processing took too long; requesting immediate shutdown.");
 pool.shutdownNow();
 pool.awaitTermination(5, TimeUnit.SECONDS);
 }
 System.out.println("Final pipeline result: " + counter.getTotal() + " records processed.");
```

```
 }
}
```

Main.java — starts the pipeline with the required values

```
package sla2;
public class Main {
 public static void main(String[] args) {
 int totalJobs = 10;
 int recordsPerJob = 1000;
 int workers = 4;
 EventTrackerEngine tracker = new EventTrackerEngine(workers);
 try {
 tracker.execute(totalJobs, recordsPerJob);
 } catch (InterruptedException e) {
 Thread.currentThread().interrupt();
 System.err.println("The pipeline was interrupted.");
 }
 }
}
```

Concepts Learned:

1. **ExecutorService and Thread Pools**
   Thread pools manage a fixed pool of worker threads to execute asynchronous tasks concurrently without the high overhead of repeatedly creating and destroying raw threads.
   • **Executors.newFixedThreadPool(n) -** Creates an ExecutorService with a fixed number of reusable worker threads backed by an unbounded queue.
   • **submit(runnable) -** Submits a Runnable task for execution and queues it if all workers are busy.
   • **shutdown() -** Initiates an orderly shutdown where previously submitted tasks are executed, but no new tasks are accepted.
   • **awaitTermination(timeout, unit) -** Blocks execution until all submitted tasks complete, a timeout occurs, or the current thread is interrupted.

• **shutdownNow()** - Attempts to stop all actively executing tasks and halts the processing of waiting tasks.

2. **Atomic Variables and Thread Safety**
   Atomic classes provide lock-free, thread-safe operations on single variables across multiple concurrent threads, preventing data races.
   • **AtomicLong** - A long value that may be updated atomically using low-level hardware support (CAS) without explicit synchronized blocks.
   • **addAndGet(delta)** - Atomically adds the specified amount to the current value and returns the updated value.
   • **get()** - Returns the current value with memory effects equivalent to reading a volatile variable.

3. **Runnable Interface and Task Delegation**
   The Runnable interface separates the definition of a task from the thread management mechanism that executes it.
   • **Runnable** - Functional interface designed for classes whose instances are intended to be executed by a thread.
   • **run()** - Contains the execution logic and batch processing computation performed by each worker.

4. **Concurrency Resource Management and Tuning**
   Proper concurrency design requires balancing system resources against workload characteristics to prevent resource exhaustion.
   • **Context Switching vs CPU-Bound** - Computation-heavy workloads align closely with available CPU cores to minimize context-switching overhead.
   • **Native Thread Exhaustion** - Unbounded raw thread creation leads to excessive memory usage and potential OutOfMemoryError failures.
   • **Thread Interruption Handling** - Preserving thread interrupt state via Thread.currentThread().interrupt() ensures graceful and responsive shutdown sequences.